Week 3

Recursion & Divide-and-Conquer – Notes

1. Recursion (Basic to Advanced)

➤ Definition

- **Recursion** is a process in which a function calls itself directly or indirectly to solve a problem.
- Every recursive function has:
  1. **Base case** – Condition where recursion stops.
  2. **Recursive case** – The part where the function calls itself with a smaller/simpler input.

How Recursion Works

- When a function is called, it is pushed onto the **call stack**.
- Each recursive call adds a new frame to the stack until the **base case** is reached.
- Then the stack **unwinds** (functions return one by one).

➤ Example: Factorial

```
def factorial(n):
    if n == 0:   # base case
        return 1
    return n * factorial(n-1)  # recursive case
```

Flow for factorial(3):

- factorial(3) → 3 * factorial(2)

- factorial(2) → 2 * factorial(1)
- factorial(1) → 1 * factorial(0)
- factorial(0) → 1 (base case)
- Unwinding: 1 → 1*1 → 2 → 6

Types of Recursion

1. **Direct Recursion** – Function calls itself directly.
2. **Indirect Recursion** – Function calls another function which eventually calls the first one.
3. **Multiple Recursion** – Function makes multiple recursive calls (e.g., Fibonacci).
4. **Mutual Recursion** – Two or more functions call each other in a cycle.

➤ Common Examples

- Factorial
- Fibonacci Sequence
- Tower of Hanoi
- Binary Search
- Merge Sort, Quick Sort
- Tree Traversals (Inorder, Preorder, Postorder)

Advantages

- Simplifies problem-solving by breaking down tasks.
- Useful for problems naturally defined recursively (trees, divide & conquer).

Disadvantages

- Higher memory usage (stack space).
- Can be slower than iterative solutions.
- Risk of **stack overflow** if recursion is too deep.

2. Tail vs Non-Tail Recursion

Tail Recursion

- A recursive function is **tail-recursive** if the **recursive call is the last operation** in the function.
- No pending operations after the recursive call.
- Compilers can optimize tail recursion into iteration (**tail call optimization**).

✅ Example (Tail Recursion – Factorial):

```
def factorial_tail(n, acc=1):
    if n == 0:
        return acc
    return factorial_tail(n-1, acc * n)
```

📌 Here, the recursive call is the **last statement**.

➤ Non-Tail Recursion

- If some computation is pending after the recursive call, it is **non-tail recursive**.
- Requires storing intermediate results in the call stack.

❌ Example (Non-Tail Recursion – Factorial):

```
def factorial_non_tail(n):
    if n == 0:
        return 1
    return n * factorial_non_tail(n-1)
```

📌 Here, multiplication happens **after** the recursive call → Not tail-recursive.

Difference Table

Aspect	Tail Recursion	Non-Tail Recursion
Last Operation	Recursive call	Recursive call + extra computation
Memory Efficiency	More efficient (can be optimized)	Less efficient (needs stack memory)
Optimization	Tail Call Optimization possible	Not possible
Example	Factorial with accumulator	Normal factorial

3. Introduction to Divide and Conquer

➤ Definition

- **Divide and Conquer** is a problem-solving paradigm where:
  1. **Divide**: Break the problem into smaller subproblems.
  2. **Conquer**: Solve the subproblems (recursively).
  3. **Combine**: Merge results to get the final answer.

General Steps

1. Identify **base case** (smallest problem that can be solved directly).
2. Divide the problem into subproblems of smaller size.
3. Solve each subproblem recursively.
4. Combine the results.

Classic Examples

- **Binary Search**
  - Divide: Split array in half.
  - Conquer: Search in the left or right half.
  - Combine: Directly return result.
  - Time Complexity: **O(log n)**
- **Merge Sort**

    - Divide: Split array into two halves.
    - Conquer: Sort each half recursively.
    - Combine: Merge two sorted halves.
    - Time Complexity: **O(n log n)**
- **Quick Sort**
    - Divide: Choose pivot and partition array.
    - Conquer: Sort left and right partitions.
    - Combine: Concatenate partitions.
    - Time Complexity: **O(n log n)** (average), **O(n^2)** (worst).
- **Matrix Multiplication (Strassen's Algorithm)**
    - Divide: Split matrices into smaller blocks.
    - Conquer: Multiply recursively.
    - Combine: Merge results.

Advantages

- Reduces problem complexity.
- Recursive nature fits many algorithmic problems.
- Often provides optimal solutions.

➤ Disadvantages

- Recursive overhead.
- May use extra memory (e.g., Merge Sort requires O(n) extra space).
- Not always the most efficient (QuickSort worst case is O(n^2)).

Examples of Recursion

1. **Factorial (Basic Recursion)**

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)

print(factorial(5))  # 120
```

✔️ Classic example of recursion.

---

2. **Fibonacci Numbers (Multiple Recursion)**

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(6))  # 8
```

✔️ Each call generates **two recursive calls**.

---

3. **Sum of Digits**

```
def sum_of_digits(n):
    if n == 0:
        return 0
    return (n % 10) + sum_of_digits(n // 10)

print(sum_of_digits(1234))  # 10
```

✔️ Breaks the number digit by digit.

---

4. **Tower of Hanoi (Advanced Recursion)**

```
def tower_of_hanoi(n, source, auxiliary, target):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    tower_of_hanoi(n-1, source, target, auxiliary)
    print(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n-1, auxiliary, source, target)

tower_of_hanoi(3, 'A', 'B', 'C')
```

✔️ Classic recursive puzzle problem.

---

5. **Reverse a String**

```
def reverse_string(s):
    if len(s) == 0:
        return ""
    return reverse_string(s[1:]) + s[0]

print(reverse_string("hello"))  # "olleh"
```

✔️ Works by breaking and combining recursively.

Examples of Divide and Conquer

1. **Binary Search**

```
def binary_search(arr, low, high, target):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
```

```
        return binary_search(arr, low, mid-1, target)
    else:
        return binary_search(arr, mid+1, high, target)

print(binary_search([1, 3, 5, 7, 9], 0, 4, 7))  # 3
```

✔️ O(log n) efficiency.

---

2. **Merge Sort**

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

✔️ Classic **divide → sort → combine** example.

---

3. **Quick Sort**

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

print(quick_sort([10, 7, 8, 9, 1, 5]))
```

✔️ Uses partitioning around a **pivot**.

---

4. **Maximum & Minimum in an Array**

```
def find_min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]
    if high == low + 1:
        return (min(arr[low], arr[high]), max(arr[low], arr[high]))

    mid = (low + high) // 2
    min1, max1 = find_min_max(arr, low, mid)
    min2, max2 = find_min_max(arr, mid+1, high)

    return min(min1, min2), max(max1, max2)

arr = [3, 5, 1, 8, 2, 7]
print(find_min_max(arr, 0, len(arr)-1))  # (1, 8)
```